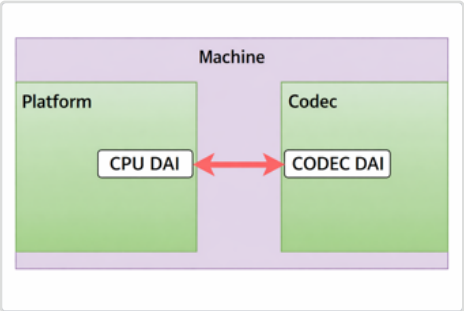
Nuvoton Audio Codec Porting and Debugging Guide

- Descriptions
  Hardware Checklist
  Device Tree Setup
  Driver Setup
  Codec Driver in Bootup
  Testing & Verification
  Troubleshooting
  Advanced Debugging

Descriptions

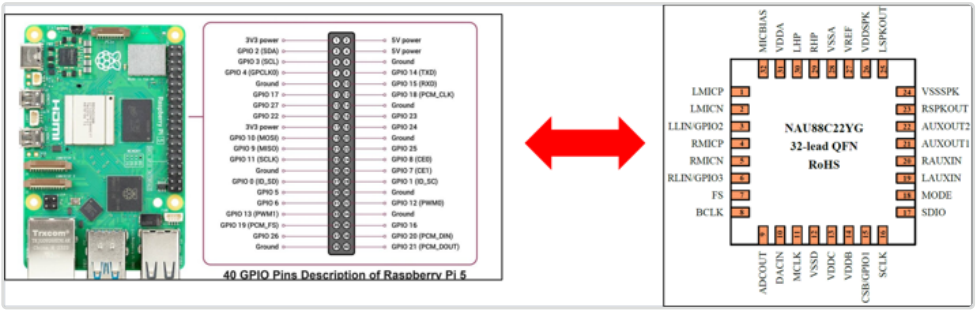
- Before porting the Nuvoton Audio Codec driver, it is essential to understand the hardware and software interaction model within the ALSA SoC (ASoC) framework.
- Based on the provided ALSA SoC architecture diagram, here are simple descriptions for the three core ASoC drivers:
  - **Machine Driver**
    - The ASoC machine driver connects the codec and platform drivers and defines the audio card configuration. Based on the Device Tree (DTS), it sets up the DAI links between the SoC CPU DAI and the codec DAI.
  - **Platform Driver**
    - The platform driver handles SoC-specific audio operations, including DMA and PCM data movement between system memory and the CPU DAI interface (e.g., I2S, TDM).
  - **Codec Driver**
    - The codec driver manages the external audio codec device. It configures codec registers, provides kcontrols (volume and mixers), manages DAPM power routing, and operates the codec DAI.



Hardware Checklist

- Please verify the following schematic connections before modifying the Device Tree.
  - Power Control

    - AVDD (Analog Power)
    - DVDD (Digital Power)
    - GND (Ground)
  - Control Interface
    - SDA/SCL (**I2C**)
    - Reset/Enable/Interrupt Pin (Optional)
  - Data Interface (**I2S**)
    - MCLK (Master Clock)
    - BCLK (Bit Clock)
    - LRCLK (Frame Sync)
    - DATA (ADCDAT / DACDAT)
- Example: (Raspberry Pi & NAU8822 Codec Schematic connection)



  - RPI GPIO 2(Pin 3 I2C1 SDA) -----⟶ NAU8822 GPIO17 (SDIO)
  - RPI GPIO 3(Pin 5 I2C1 SCL)) -----⟶ NAU8822 GPIO16 (SCLK
  - RPI GPIO 18(Pin 12 PCM CLK) -----⟶ NAU8822 GPIO8 (BCLK)
  - RPI GPIO 19(Pin 35 PCM FS) -----⟶ NAU8822 GPIO7 (FS)
  - RPI GPIO 20(Pin 38 PCM DIN) -----⟶ NAU8822 GPIO9 (ADCOUT)
  - RPI GPIO 21(Pin 40 PCM DOUT) -----⟶ NAU8822 GPIO10 (DACIN)
  - RPI GPIO 4(Pin 7 GPCLK0) -----⟶ NAU8822 GPIO11 (MCLK)
  - RPI (Pin 39 Ground) -----⟶ NAU8822 GPIO12 (VSSD)

Device Tree Setup

- Now that the hardware connections are established, the next step is to write a **Device Tree Overlay (DTS)**. The Device Tree setup tailored to your specific pin connections (including I2C control, I2S data transfer, and the MCLK).
- **Generic DTS Example:**
  - Machine Driver (Sound Card) Node & DAI Binding

```
/ {
    /* Define the sound card at the root level */
    sound {
```

```
        /* (1) Binds to the specific Machine Driver's of_match_table */
        compatible = "nuvoton,pisound-nau8822";

        /* (2) Binds the CPU DAI (Reference to the SoC's I2S/SAI interface) */
        i2s-controller = <&i2s>;

        /* (3) MCLK hardware source (Replace with the SoC's actual clock provider node) */
        clocks = <&mclk_provider_node>;
        clock-names = "mclk";

        /* (4) Target MCLK frequency passed to the C driver to configure sysclk */
        nuvoton,clock-rates = <12288000>;

        status = "okay";
    };
};
```

- I2C Bus Configuration & Slave Address

```
/* Replace &i2c_bus with the actual I2C node of the SoC (e.g., &i2c1, &i2c2) */
&i2c_bus {
    status = "okay";

    nau8822: codec@1a {
        compatible = "nuvoton,nau8822";
        reg = <0x1a>;  /* This is the I2C Slave Address for NAU8822 */

        /* Declares this node as an Audio DAI (Digital Audio Interface) device */
        #sound-dai-cells = <0>;
    };
};
```

- MCLK (Master Clock) Configuration
  - Simplified Clock Relationship

    To maintain perfect synchronization, the BCLK is derived from the MCLK via a hardware divider (***BCLK = MCLK / x***).
  - **MCLK (Master Clock)**: The **Master Clock (MCLK)** serves as the primary timing reference. It is usually calculated as ***Sample Rate x 256***. For a 48 kHz sample rate, MCLK is 12.288 MHz.
  - **BCLK (Bit Clock)**: Synchronizes each individual data bit on the I2S bus. It is calculated as ***Sample Rate x Channels x Bitsper Sample***. For a 2-channel, 32-bit stream at 48 kHz, BCLK is 3.072 MHz.
  - ```
    / {
        nau8822_mclk: codec_mclk {
            compatible = "fixed-clock";
            #clock-cells = <0>;
            clock-frequency = <12288000>; /* Define the actual MCLK input frequency */
        };
    };
    ```

Driver Setup

- 1. Driver Porting: **Machine Driver (Board-level)** Implementation
    ```

- The Machine Driver serves as the "glue" that binds generic drivers (Codec and Platform) together. Its primary purpose is to manage the unique Hardware Characteristics specific to a particular development board.
- Key Hardware Characteristics Include:
  - DAI Links: Defining the specific communication bridge between the CPU and the Codec.
  - External Component Control: Managing board-level logic, such as enabling an external Power Amplifier (PA) via GPIO or handling Jack Detection (plug-in/plug-out) interrupts.
  - Clock Configuration: Determining the reference clock source for I2S synchronization, such as an onboard MCLK or an External Oscillator (OSC).
- The Four Primary Responsibilities
  - **Populating the `snd_soc_dai_link` Structure**: Explicitly mapping the appropriate CPU DAI to the corresponding Codec DAI to establish the data path.
  - **Clock and Master/Slave Configuration**: Setting up physical clock frequencies (MCLK) and defining whether the SoC or the Codec operates as the I2S **Master** or **Slave**.
  - **Defining DAPM Widgets and Routing**: Establishing the audio routes between physical board connectors (e.g., Speakers, Mic Jacks) and the internal logic of the Codec to complete the **DAPM (Dynamic Audio Power Management) path**.
  - **Runtime Parameter Propagation**: Ensuring that dynamic stream data, such as the **Sampling Frequency**, is correctly propagated to all individual driver components at runtime.
- Machine Driver Implementation Requirements:
  - Step1: Create Platform Driver
    - The `compatible` **string** in your Platform Driver's match table must be **exactly identical** to the one defined in your Device Tree sound node to successfully trigger the driver's probe function.

```c
static const struct of_device_id pisound_nau8822_of_match[] = {
    { .compatible = "nuvoton,pisound-nau8822", },
    {},
};
MODULE_DEVICE_TABLE(of, pisound_nau8822_of_match);

static struct platform_driver pisound_nau8822_driver = {
    .driver = {
        .name   = "snd-pisound-nau8822",
        .owner  = THIS_MODULE,
        .of_match_table = pisound_nau8822_of_match,
    },
    .probe = pisound_nau8822_probe,
    .remove = pisound_nau8822_remove,
};
module_platform_driver(pisound_nau8822_driver);
```

  - Step2: Sound Card Registration
    - **Create Sound Card (`struct snd_soc_card`)**: Define the structure that represents the logical audio device, including its name and hardware properties.

```
static struct snd_soc_card snd_soc_pisound_nau8822 = {
    .name = "pisoundnau8822",
    .owner = THIS_MODULE,
    .dai_link = pisound_nau8822_dai,
    .num_links = ARRAY_SIZE(pisound_nau8822_dai),

    .dapm_widgets = pisound_nau8822_dapm_widgets,
    .num_dapm_widgets = ARRAY_SIZE(pisound_nau8822_dapm_widgets),
    .dapm_routes = pisound_nau8822_audio_map,
    .num_dapm_routes = ARRAY_SIZE(pisound_nau8822_audio_map),
};
```

- **DAI Configuration (`struct snd_soc_dai_link`)**: Set up the Digital Audio Interface (DAI) links to define the physical data path between the SoC's I2S controller (SoC) and the NAU8822 (Codec).

```
static struct snd_soc_dai_link pisound_nau8822_dai[] = {
    {
        .name = "pisound nau8822",
        .stream_name = "pisound nau8822",
        .ops = &pisound_nau8822_ops,
        .init = pisound_nau8822_dai_init,
#if 1 // slave mode
        .dai_fmt = SND_SOC_DAIFMT_I2S | SND_SOC_DAIFMT_NB_NF | SND_SOC_DAIFMT_CBS_CFS,
#else // master mode
        .dai_fmt = SND_SOC_DAIFMT_I2S | SND_SOC_DAIFMT_NB_NF | SND_SOC_DAIFMT_CBM_CFM,
#endif
        SND_SOC_DAILINK_REG(nau8822_link),
    },
};
```

```
/* machine stream operations */
static struct snd_soc_ops pisound_nau8822_ops = {
    .startup = pisound_nau8822_startup,
    .hw_params = pisound_nau8822_hw_params,
    .shutdown = pisound_nau8822_shutdown,
};
```

- **Register the Sound Card**: Invoke the ASoC registration API (e.g., `devm_snd_soc_register_card`) to bind the CPU, Codec, and Platform components. This process instantiates the sound card, making it visible to the ALSA subsystem and accessible to user-space applications (such as `aplay` or `amixer`).

```
static int pisound_nau8822_probe(struct platform_device *pdev)
{
    struct snd_soc_card *card = &snd_soc_pisound_nau8822;
    struct pisound_nau8822_dev *snd_nau8822;
    int ret;

    dev_dbg(&pdev->dev, "%s...", __func__);

    card->dev = &pdev->dev;
    snd_nau8822 = devm_kzalloc(&pdev->dev, sizeof(*snd_nau8822), GFP_KERNEL);
    if (!snd_nau8822)
        return -ENOMEM;
    snd_nau8822->card = card;

    if (pdev->dev.of_node) {
        struct snd_soc_dai_link *dai = &pisound_nau8822_dai[0];
        struct device_node *i2s_node = of_parse_phandle(pdev->dev.of_node,
                                                        "i2s-controller", 0);
        if (i2s_node) {
            dai->cpus->dai_name = NULL;
            dai->cpus->of_node = i2s_node;
            dai->platforms->name = NULL;
            dai->platforms->of_node = i2s_node;
        } else if (!dai->cpus->of_node) {
            dev_err(&pdev->dev, "Property 'i2s-controller' missing or invalid\n");
            return -EINVAL;
        }
    }
    snd_soc_card_set_drvdata(card, snd_nau8822);
    ret = devm_snd_soc_register_card(&pdev->dev, &snd_soc_pisound_nau8822);
    if (ret && ret != -EPROBE_DEFER)
        dev_err(&pdev->dev, "Failed to register card %d\n", ret);
```

- Step3: Machine DAI Configuration (DAI Initial)
  - a. dai_link configuration



- b. dai_fmt configuration

```
#if 1 // slave mode
        .dai_fmt = SND_SOC_DAIFMT_I2S | SND_SOC_DAIFMT_NB_NF | SND_SOC_DAIFMT_CBS_CFS,
#else // master mode
        .dai_fmt = SND_SOC_DAIFMT_I2S | SND_SOC_DAIFMT_NB_NF | SND_SOC_DAIFMT_CBM_CFM,
#endif
        SND_SOC_DAILINK_REG(nau8822_link),
    },
};
```

- c. dapm_widgets configuration

```
static const struct snd_soc_dapm_widget pisound_nau8822_dapm_widgets[] = {
    SND_SOC_DAPM_HP("Headphones", NULL),
    SND_SOC_DAPM_MIC("Microphone", NULL),
};
```

- d. dapm_routes configuration

```
static const struct snd_soc_dapm_route pisound_nau8822_audio_map[] = {
    { "Headphones", NULL, "LHP" },
    { "Headphones", NULL, "RHP" },
    { "LMICN", NULL, "Microphone" },
    { "LMICP", NULL, "Microphone" },
};
```

- e. snd_soc_ops configuration

```
static int ma35d1_audio_hw_params(struct snd_pcm_substream *substream, struct snd_pcm_hw_params *params)
{
        unsigned int fmt = SND_SOC_DAIFMT_I2S | SND_SOC_DAIFMT_NB_NF | SND_SOC_DAIFMT_CBS_CFS;
        struct snd_soc_pcm_runtime *rtd = substream->private_data;
        struct snd_soc_dai *codec_dai = asoc_rtd_to_codec(rtd, 0);
        struct snd_soc_dai *cpu_dai = asoc_rtd_to_cpu(rtd, 0);
        struct ma35d1_audio *ma35d1_audio = dev_get_drvdata(cpu_dai->dev);
        unsigned int cpu_mclk;
        int ret;

        /* set codec DAI configuration */
        ret = snd_soc_dai_set_fmt(codec_dai, fmt);
        if (ret < 0)
                return ret;

        /* set cpu DAI configuration */
        ret = snd_soc_dai_set_fmt(cpu_dai, fmt);
        if (ret < 0)
                return ret;

        cpu_mclk = ma35d1_audio->mclk_out;

        /* set the codec system clock */
        ret = snd_soc_dai_set_sysclk(codec_dai, NAU8822_CLK_MCLK,
                                     cpu_mclk, SND_SOC_CLOCK_IN);
        if (ret < 0 )
                return ret;

        return 0;
}

static struct snd_soc_ops ma35d1_audio_ops = {
        .hw_params = ma35d1_audio_hw_params,
};
```

- Step4: Jack Detection (Optional)
  - Jack Detection is managed by the Machine Driver to monitor the physical state of the audio connectors. This allows the system to notify user-space applications and trigger DAPM (Dynamic Audio Power Management) switches automatically.
  - 1. Create a Jack Instance (`snd_soc_jack`)

- `snd_soc_jack` : Represents the logical jack instance.
- `snd_soc_jack_pin` : Connects the jack instance to a specific DAPM pin (endpoint), such as "Headphones".

```
static struct snd_soc_jack pisound_headphone;
static struct snd_soc_card snd_soc_pisound_nau8822;

static struct snd_soc_jack_pin pisound_jack_pins[] = {
    {
        .pin    = "Headphones",
        .mask   = SND_JACK_HEADPHONE,
    },
};
```

- 2. Register the Jack to the Sound Card
  - Use `snd_soc_card_jack_new_pins()` to initialize the jack within the machine driver's scope.

```
ret = snd_soc_card_jack_new_pins(&snd_soc_pisound_nau8822, "Headphones",
                                 SND_JACK_HEADPHONE, &pisound_headphone,
                                 pisound_jack_pins,
                                 ARRAY_SIZE(pisound_jack_pins));
if (ret) {
    dev_err(rtd->dev, "Headphone Jack creation failed %d\n", ret);
    return ret;
}
```

- 3. Configure GPIO for Detection
  - To link a physical GPIO pin to the jack detection logic, define a `snd_soc_jack_gpio` structure and register it using `snd_soc_jack_add_gpios()`.
  - When the GPIO state changes, the kernel executes an interrupt function to update the pin status and report it to user-space via `snd_soc_jack_report()`

```
static struct snd_soc_jack_gpio pisound_hp_jack_gpio= {
    .gpio = GPIO4_FOR_HPDET,
    .name = "Headphone detection",
    .report = SND_JACK_HEADPHONE,
    .debounce_time = 500,
    .invert = 1,//GPIO_ACTIVE_LOW
};
```

```
ret = snd_soc_jack_add_gpios(&pisound_headphone, 1, &pisound_hp_jack_gpio);
if (ret) {
    dev_err(rtd->dev, "Add jack gpio failed %d\n", ret);
    return ret;
}
dev_info(rtd->dev, "Headphone Jack creation is done\n");
```

- 2. Driver Porting: **Platform Driver (SoC-level)** Implementation
  - The platform driver primarily focuses on the I2S/PCM controller and the DMA engine.
  - The platform driver registers the PCM driver, CPU DAI driver, and their operation functions, pre-allocates buffers for PCM components, and sets playback and capture operations as applicable.
  - Platform Driver Implementation Requirements:
    - Step1: Create Platform Driver
      - Like the Machine Driver, the Platform Driver uses a standard Linux `platform_driver` structure for kernel integration:

```c
static const struct of_device_id ma35d1_audio_pcm_of_match[] = {
        {   .compatible = "nuvoton,ma35d1-audio-pcm"    },
        {   },
};
MODULE_DEVICE_TABLE(of, ma35d1_audio_pcm_of_match);

static struct platform_driver ma35d1_pcm_driver = {
        .driver = {
                .name = "ma35d1-audio-pcm",
                .owner = THIS_MODULE,
                .of_match_table = of_match_ptr(ma35d1_audio_pcm_of_match),
        },

        .probe = ma35d1_soc_platform_probe,
        .remove = ma35d1_soc_platform_remove,
};

module_platform_driver(ma35d1_pcm_driver);
```

- Step2: PCM Hardware Configuration (`snd_pcm_hardware`)
  - This structure describes the SoC's audio capabilities.
    - **Formats**: Bit depths the SoC can process (e.g., 16-bit or 24-bit Little Endian).
    - **Rates**: Supported sampling frequencies.
    - **Buffer Sizes**: Maximum and minimum memory allocation for audio playback and capture.

```c
#define MA35D1_DMABUF_SIZE      (64 * 1024)

static const struct snd_pcm_hardware ma35d1_pcm_hardware = {
        .info           = SNDRV_PCM_INFO_INTERLEAVED |
        SNDRV_PCM_INFO_BLOCK_TRANSFER |
        SNDRV_PCM_INFO_MMAP |
        SNDRV_PCM_INFO_MMAP_VALID |
        SNDRV_PCM_INFO_PAUSE |
        SNDRV_PCM_INFO_RESUME,
        .formats        = SNDRV_PCM_FMTBIT_S16_LE |
        SNDRV_PCM_FMTBIT_S24_LE,
        .rates          = SNDRV_PCM_RATE_32000 |
        SNDRV_PCM_RATE_44100 |
        SNDRV_PCM_RATE_48000,
        .channels_min       = 1,
        .channels_max       = 2,
        .buffer_bytes_max   = 64*1024,
        .period_bytes_min   = 4*1024,
        .period_bytes_max   = 8*1024,
        .periods_min        = 1,
        .periods_max        = 1024,
};

static const struct snd_dmaengine_pcm_config ma35d1_dmaengine_pcm_config = {
        .pcm_hardware = &ma35d1_pcm_hardware,
        .prepare_slave_config = snd_dmaengine_pcm_prepare_slave_config,
        .prealloc_buffer_size = 32 * 1024,
};
```

- Step3: DMA Engine Config (`snd_dmaengine_pcm_config`)
  - This links the ALSA PCM core to the Linux DMA engine. It ensures that the `prepare_slave_config` and `pcm_hardware` settings are applied to the DMA transfer.

```c
static const struct snd_dmaengine_pcm_config ma35d1_dmaengine_pcm_config = {
        .pcm_hardware = &ma35d1_pcm_hardware,
        .prepare_slave_config = snd_dmaengine_pcm_prepare_slave_config,
        .prealloc_buffer_size = 32 * 1024,
};
```

- Step4: DMA Registration
  - `devm_snd_dmaengine_pcm_register` is a core API within the Linux ASoC framework, primarily used during the platform driver initialization.
  - Its main purpose is to bridge the SoC DMA engine with the ALSA PCM core, enabling efficient audio data transfer between system memory and the I2S/PCM controller.

```c
static int ma35d1_soc_platform_probe(struct platform_device *pdev)
{
        return devm_snd_dmaengine_pcm_register(&pdev->dev, &ma35d1_dmaengine_pcm_config,
                                               SND_DMAENGINE_PCM_FLAG_COMPAT);
}
```

- Kconfig Configuration (for example: MA35D1)
  - This section enables the low-level I2S and DMA controllers of the SoC (e.g., Nuvoton MA35D1).

```
config SND_SOC_MA35D1
        tristate "SoC Audio for MA35D1 series"
        depends on ARCH_MA35D1
        help
          This option enables support for I2S on the MA35D1 SoC.

##
## Boards
##
config SND_SOC_MA35D1EVB
        tristate "MA35D1 I2S support for demo board"
        depends on SND_SOC_MA35D1
        select SND_SOC_NAU8822
        select SND_SOC_GENERIC_DMAENGINE_PCM
        help
          Select this option to enable audio (I2S) on the
          MA35D1 demoboard.
```

- Makefile Definition
  - `ma35d1-pcm.o` : Manages the **DMA Engine**, which handles the data transfer between system memory and the audio hardware.
  - `ma35d1-i2s.o` : Manages the **Digital Audio Interface (DAI)**, which controls the physical pins and timing (MCLK/BCLK).
  - `CONFIG_SND_SOC_MA35D1` : If this SoC support is enabled in the kernel config, both the I2S and PCM objects are compiled and linked.

```
snd-soc-ma35d1-pcm-objs := ma35d1-pcm.o
snd-soc-ma35d1-i2s-objs := ma35d1-i2s.o
snd-soc-ma35d1-audio-objs := ma35d1-audio.o

obj-$(CONFIG_SND_SOC_MA35D1) += snd-soc-ma35d1-i2s.o snd-soc-ma35d1-pcm.o snd-soc-ma35c

```

- 3. Driver Porting: **Codec Driver**
  - Driver Integration (Kbuild System)
    - To integrate the **NAU8xxx** into the Linux kernel, you must modify the configuration and build files within the `sound/soc/codecs/` directory.
      - Kconfig Configuration
        - The `Kconfig` file allows the driver to appear in the `menuconfig` interface. Add the following entry to `sound/soc/codecs/Kconfig` :

```
config SND_SOC_NAU8xxx
    tristate "Nuvoton NAU8xxx codec support"
    depends on I2C
    help
      Enable support for the Nuvoton NAU8xxx audio codec.
```

      - Makefile Definition
        - The `Makefile` tells the build system which source files to compile. Add these lines to `sound/soc/codecs/Makefile` :

- `CONFIG_SND_SOC_NAU8xxx` : If this Codec support is enabled in the kernel config, the NAU8xxx object files will be compiled and linked into the kernel or as a loadable module.

```
snd-soc-nau8xxx-objs := nau8xxx.o
obj-$(CONFIG_SND_SOC_NAU8xxx) += snd-soc-nau8xxx.o
```

Codec Driver in Bootup

- Verifying Codec Driver Probe and Sound Card Status in Bootup. (For example: nau8822 codec driver)
  - Step 1: Check the NAU8822 Codec Driver Probe (I2C Detection)
    - During the system boot sequence, the Linux kernel initializes devices based on the Device Tree. Checking the kernel logs using `dmesg` is the most effective way to confirm that the NAU8822 codec and the ASoC machine driver have probed successfully, or to identify where a failure occurred.
    - Run the following command after boot:

```
dmesg | grep -i nau8822
```

    - Expected Output:
      - Kernel log: You should see a message indicating the codec was registered.

```
[    1.452300] nau8822 1-001a: NAU8822 Audio Codec probed successfully
```

      - (Note: 1-001a refers to I2C bus 1, Slave Address 0x1a).
  - Step 2: Check the Machine Driver and Sound Card Registration
    - After the codec and the SoC's I2S (CPU DAI) drivers are loaded, the Machine Driver attempts to bind them together and register the logical sound card.
    - Command:

```
dmesg | grep -i sound
```

    - Kernel log:

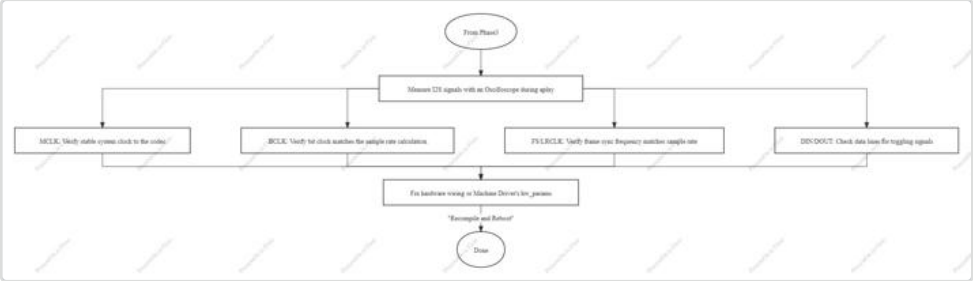
```
[    2.105400] asoc-simple-card sound: Custom-NAU8822-Card mapping ok
# or, if using a dedicated machine driver:
[    2.105400] nuvoton,pisound-nau8822 sound: SoC I2S <-> nau8822-hifi mapping ok
```

Testing & Verification

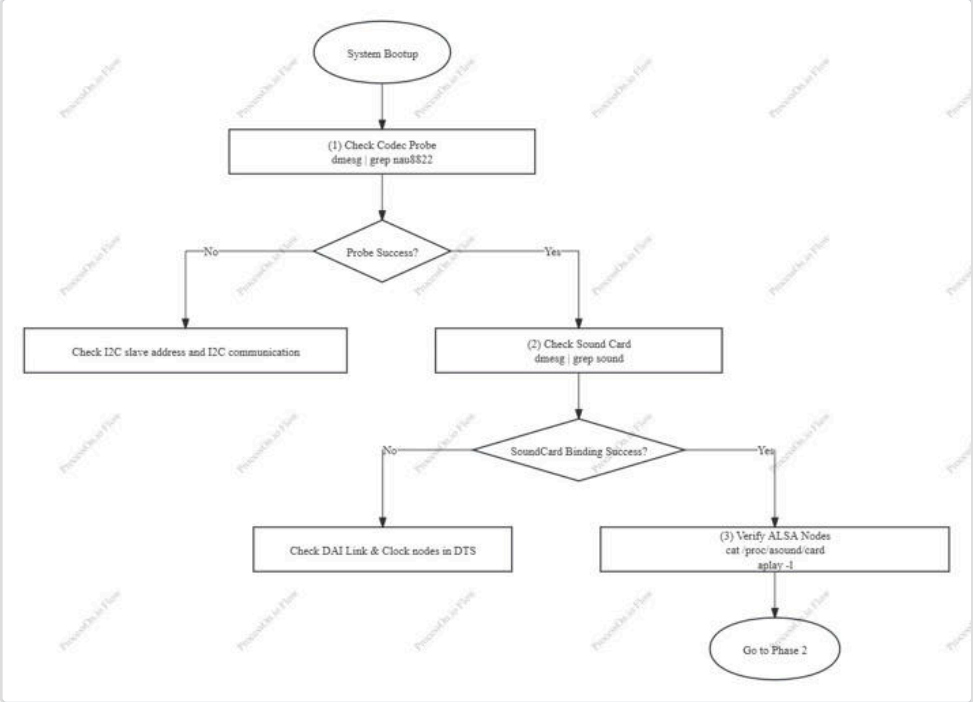
- **Hardware** Verification
  - Checking I2C Signal



- Checking I2S/PCM Signal



- **Software** Verification
  - **System Init & Registration**



  - **Playback Test (Playback):**
    - Command: `aplay -l` to list the card.
    - Command: `aplay -D hw:X,Y test.wav` (where X is card index).
  - **Record Test (Capture):**
    - Command: `arecord -D hw:X,Y -f S16_LE -r 44100 record.wav`.
  - **Mixer & Path Verification:**
    - Use `alsamixer` or `amixer` to toggle switches and adjust volumes.

- Confirm that the Headphone Jack or Speaker output behaves according to the mixer settings.
- Dump All Controls and Current Values
  - Run this command to see every "kcontrol" (Kernel Control) registered by the driver, including volumes, muxes, and switches.

```
amixer -c 0 contents
```

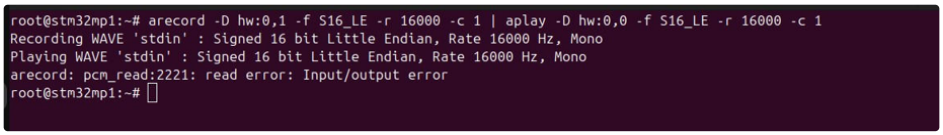
Troubleshooting

- **Issue 1: Device Not Found (I2C Probe Failure)**
  - **Symptom:** `dmesg` shows I2C timeout or `nau8xxx: probe failed`.
  - **Action:**
    - 1. Run `i2cdetect -y <i2c_bus_number>` to verify the codec responds at its I2C address.
    - 2. If the address is missing, check the 3.3V/1.8V power supply and the I2C pull-up resistors.
    - 3. Confirm that the compatible property in the i2c codec node exactly matches the `of_match_table` defined in the codec driver source code (e.g., `compatible = "nuvoton,nau8xxx";`). If there is a typo or mismatch in this string, the kernel will completely ignore the device and will not trigger the driver's probe function.
- **Issue 2: Sound Card Registers Failure**
  - **Symptom:** The sound card is not recognized by the system. Running `aplay -l`. Sound card is completely missing from the list
  - **Action:**
    - Verify Machine Driver I2S Interface Name (CPU DAI)



    - Check the `snd_soc_dai_link` array in your Machine Driver source code.
      - `COMP_CPU` **&** `COMP_PLATFORM` **:** Both macros define the host side of the connection. They must be set exactly to the registered I2S controller name
      - `COMP_CODEC` **:** This macro requires two specific strings to locate the Nuvoton codec:
        - **Driver Name (** `"nau8822.1-001a"` **):** This string represents the physical device path. The `1` represents the I2C bus number, and `001a` represents the hexadecimal I2C slave address (0x1A).

    - **DAI Name (`"nau8822-hifi"`):** This is the logical Digital Audio Interface name exposed by the codec driver. It must match the string defined in the `snd_soc_dai_driver` struct inside the codec's source code (e.g., `nau8822.c`).
- **Issue 3: No Audio Output**
    - **Symptom:** This error occurs during audio recording (capture) or playback. The hardware fails to generate the necessary I2S clocks, resulting in both the Bit Clock (BCLK) and Frame Sync (Fs / LRCK) lines showing absolutely no output on an oscilloscope.



    - **Action:**
        - **1. Incorrect Soc pinmux configuration:**
            - Verify the `pinctrl` (pin multiplexing) settings in your Device Tree. Ensure that the SoC pins designated for BCLK, Fs, Data In, and Data Out are configured for their specific I2S alternate functions, rather than defaulting to standard GPIOs.
        - **2. Incorrect MCLK setting:**
            - Check the Master Clock (MCLK) frequency. If the Codec's internal PLL or the host SoC does not receive the expected base clock (e.g., 12.288 MHz), verify the `clocks` property in the Device Tree and the `snd_soc_dai_set_sysclk()` configuration in the Machine Driver.
        - **3. No BCLK output:**
            - Check the Master/Slave role configuration in the Machine Driver (dai_fmt). If both are set as slaves, no clock will be generated.
        - **4. No Frame Sync(Fs) output:**
            - If BCLK is present but Fs is missing, check the sample rate constraints in the driver's `hw_params` function. A miscalculation in the hardware divider or an unsupported sample rate will prevent the Frame Sync pulse from being generated.
- **Issue 4: UCM (Use Case Manager) Load Failure**
    - **Symptom:** The initial settings defined in the UCM configuration (e.g., default volume levels, unmute states, or default routing) are not applied upon boot or when the sound card is selected.
    - **Action:**
        - Run the following command to check whether the UCM configuration files are being loaded successfully.
            - `strace -f -e openat alsaucm -c hw:2 dump text 2>&1 | grep -E "ucm2|HiFi\.conf"`

```
pi@raspberrypi:~ $ strace -f -e openat alsaucm -c hw:2 dump text 2>&1 | grep -E "ucm2|HiFi\.conf"
openat(AT_FDCWD, "/usr/share/alsa/ucm2/ucm.conf", O_RDONLY) = 4
openat(AT_FDCWD, "/usr/share/alsa/ucm2/lib/generic.conf", O_RDONLY) = 4
openat(AT_FDCWD, "/usr/share/alsa/ucm2/conf.d/pisoundnau8360/pisoundnau8360.conf", O_RDONLY) = 4
openat(AT_FDCWD, "/usr/share/alsa/ucm2/conf.d/pisoundnau8360/HiFi.conf", O_RDONLY) = 4
pi@raspberrypi:~ $
```

- **Issue 5: Complex Configuration Issues (Register Dump & Comparison)**
  - **Symptom:** The audio behaves unexpectedly (e.g., incorrect audio routing, volume anomalies, or missing hardware features), and standard kernel logs (dmesg) do not indicate any obvious errors. A hardware register comparison is required to verify the codec's actual internal state.
  - **Action:** Dump All Registers status (1-0021: <i2c bus id> - <slave address>)
    - sudo cat /sys/kernel/debug/regmap/1-0021/registers

```
pi@raspberrypi:~/work $
pi@raspberrypi:~/work $ sudo cat /sys/kernel/debug/regmap/1-0021/registers
00: 0000
01: 0000
02: 21f2
03: 1000
04: 0000
05: 007f
06: 0000
09: 0000
0a: 0000
0b: 0000
0c: 0000
0d: 0002
0e: 0000
0f: 0000
10: 0000
11: 0000
12: 8000
13: f3f3
1d: 0000
1f: 0000
22: 1f80
29: 0082
2a: 0000
```

Advanced Debugging

- **DAPM traces**
  - The Linux kernel provides the `tracefs` filesystem as part of its tracing framework, allowing developers to monitor the internal behavior of the **ALSA** and **ASoC** subsystems in real-time. These tracepoints are essential for tracking the execution flow of audio events, such as DAPM power transitions and I2S stream state changes.
    - 1. Clear trace buffer

```
echo 0 | sudo tee /sys/kernel/debug/tracing/trace
```

    - 2. Enable event tracepoint

```
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_widget_event_start
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_widget_event_done/
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_connected/enable
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_done/enable
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_path/enable
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_start/enable
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_walk_done/enable
echo 1 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_widget_power/enabl
```

    - 3. Play wav file

```
aplay hotel_california.wav
```

- 4. Disable event tracepoint

```
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_widget_event_start
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_widget_event_done/
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_connected/enable
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_done/enable
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_path/enable
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_start/enable
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_walk_done/enable
echo 0 | sudo tee /sys/kernel/debug/tracing/events/asoc/snd_soc_dapm_widget_power/enabl
```

- 5. Dump trace buffer

```
sudo cat /sys/kernel/debug/tracing/trace
```

```
pi@raspberrypi:~/work $ sudo cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
# entries-in-buffer/entries-written: 69/69   #P:4
#
#                              _-----=> irqs-off/BH-disabled
#                             / _----=> need-resched
#                            | / _---=> hardirq/softirq
#                            || / _--=> preempt-depth
#                            ||| / _-=> migrate-disable
#                            |||| /     delay
#           TASK-PID     CPU#  |||||  TIMESTAMP  FUNCTION
#              | |         |   |||||     |         |
       pipewire-942     [002] .....   402.337238: snd_soc_dapm_start: card=pisoundnau8360
       pipewire-942     [002] .....   402.337242: snd_soc_dapm_widget_power: widget=Sense val=1
       pipewire-942     [002] .....   402.337243: snd_soc_dapm_widget_power: widget=Playback val=1
       pipewire-942     [002] .....   402.337245: snd_soc_dapm_path: *ADC_V <- (direct) <- Sense
       pipewire-942     [002] .....   402.337245: snd_soc_dapm_widget_power: widget=ADC_V val=1
       pipewire-942     [002] .....   402.337246: snd_soc_dapm_path: *ADC_I <- (direct) <- Sense
       pipewire-942     [002] .....   402.337246: snd_soc_dapm_widget_power: widget=ADC_I val=1
       pipewire-942     [002] .....   402.337247: snd_soc_dapm_path: *AIFRX <- (direct) <- Playback
       pipewire-942     [002] .....   402.337247: snd_soc_dapm_widget_power: widget=AIFRX val=1
       pipewire-942     [002] .....   402.337248: snd_soc_dapm_path: *HW3 Engine <- (direct) <- ADC_V
       pipewire-942     [002] .....   402.337248: snd_soc_dapm_path: *HW3 Engine <- (direct) <- ADC_I
       pipewire-942     [002] .....   402.337248: snd_soc_dapm_widget_power: widget=HW3 Engine val=1
       pipewire-942     [002] .....   402.337249: snd_soc_dapm_path: *HW1 Engine <- (direct) <- AIFRX
       pipewire-942     [002] .....   402.337249: snd_soc_dapm_widget_power: widget=HW1 Engine val=1
```